GRAFO DE CONOCIMIENTO EN NEO4J – EDUSMART

4.8.1. Construcción del grafo de conocimiento

Para el proyecto EduSmart se propone la representación de una parte significativa de la ontología del dominio mediante Neo4j, un sistema de gestión de bases de datos orientado a grafos que permite organizar información a través de nodos y relaciones.

El propósito de esta representación es identificar las conexiones entre los estudiantes, sus necesidades académicas, los recursos educativos, los asesores y los servicios de acompañamiento. De esta manera, se facilita la consulta de información y la generación de recomendaciones a partir de las relaciones existentes.

4.8.2. Nodos implementados

Se diseñaron diez nodos correspondientes a diferentes elementos del dominio educativo: Estudiante, Asesor, Administrador, NecesidadAcademica, RecursoEducativo, Categoria, Tema, Recomendacion y ServicioAcademico. Para la representación se incluyeron dos individuos de tipo RecursoEducativo.

Cada nodo contiene propiedades descriptivas que permiten identificarlo y caracterizarlo dentro del sistema. Por ejemplo, los estudiantes incluyen información sobre su nombre y programa académico, mientras que los recursos educativos contienen su nombre, nivel de dificultad y disponibilidad.

4.8.3. Relaciones implementadas

Se establecieron catorce relaciones entre los nodos, distribuidas en doce tipos de relaciones diferentes. Entre ellas se encuentran PRESENTA_NECESIDAD, ORIENTA_A, OFRECE_SERVICIO, ATIENDE_NECESIDAD, PERTENECE_A_CATEGORIA, ABORDA_TEMA, RESPONDE_A_NECESIDAD, RECOMIENDA_RECURSO, RELACIONADO_CON_TEMA, TIENE_INTERES_EN, TIENE_TEMA y ORIENTA_RECURSO.

Estas relaciones permiten representar cómo interactúan los diferentes elementos de EduSmart y cómo se conectan las necesidades académicas con los recursos disponibles.

4.8.4. Consultas desarrolladas en Cypher

Se diseñaron tres consultas en lenguaje Cypher para responder a necesidades específicas del proyecto:

Consulta 1. Identificación de necesidades académicas: permite consultar las necesidades registradas por un estudiante, mostrando su identificación, programa académico y descripción de la necesidad.

Consulta 2. Búsqueda de recursos educativos: permite identificar los recursos que se encuentran relacionados directamente con una necesidad académica, incluyendo su nivel de dificultad y disponibilidad.

Consulta 3. Generación de recomendaciones mediante relaciones indirectas: permite identificar recursos educativos que abordan el mismo tema asociado con una necesidad del estudiante, aunque no exista una relación directa previamente registrada entre el recurso y dicha necesidad.

4.8.5. Generación de conocimiento derivado

La tercera consulta constituye un ejemplo de conocimiento derivado, debido a que utiliza las relaciones existentes entre estudiantes, necesidades académicas, temas y recursos educativos para identificar una posible recomendación adicional.

Por ejemplo, cuando un estudiante presenta dificultades en programación y su necesidad está relacionada con los fundamentos de JavaScript, el sistema puede identificar otros recursos educativos que aborden ese mismo tema. De esta manera, se obtiene una recomendación candidata que no estaba registrada directamente como una relación entre el recurso y la necesidad.

Esta consulta demuestra cómo el grafo puede aprovechar las conexiones entre sus elementos para descubrir asociaciones útiles para la orientación académica.

4.8.6. Relación entre la ontología y el grafo de conocimiento

La representación en Neo4j se fundamenta en las clases y propiedades definidas previamente en Protégé. Las clases se representan como etiquetas de nodos, mientras que las propiedades de objeto se transforman en relaciones dirigidas entre dichos nodos.

Esta correspondencia permite mantener la coherencia conceptual entre la ontología y el grafo de conocimiento. Asimismo, facilita la consulta de información y la identificación de relaciones relevantes para las funcionalidades de EduSmart.

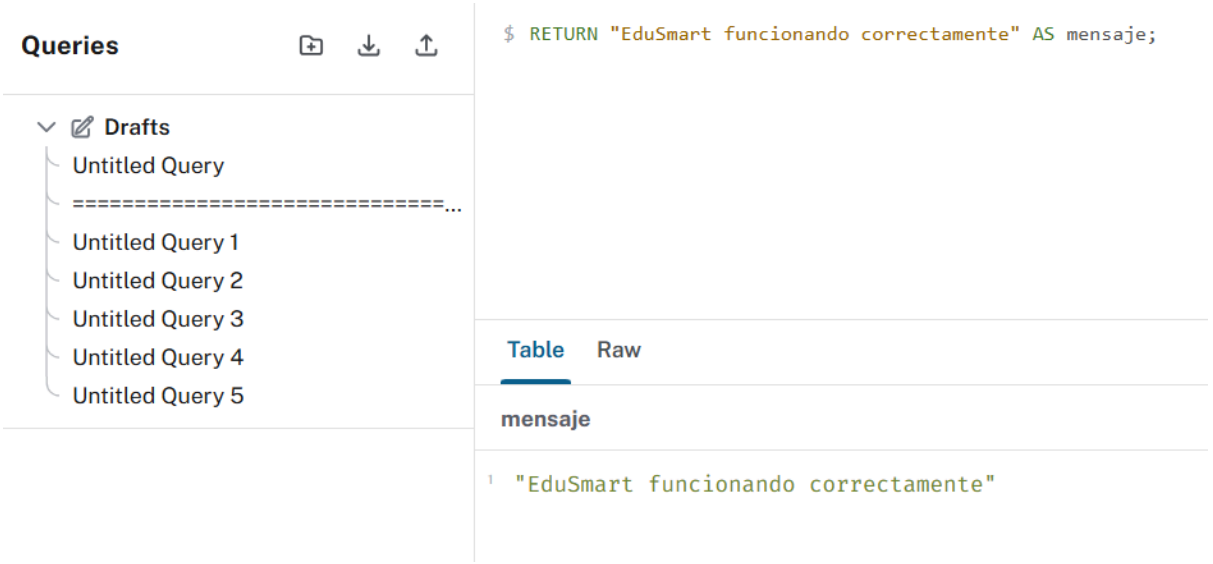
4.8.7. Resultado esperado

Con la construcción del grafo se busca demostrar que EduSmart puede representar las relaciones entre sus principales elementos, consultar las necesidades académicas de los estudiantes, identificar recursos educativos y generar recomendaciones a partir de asociaciones temáticas.

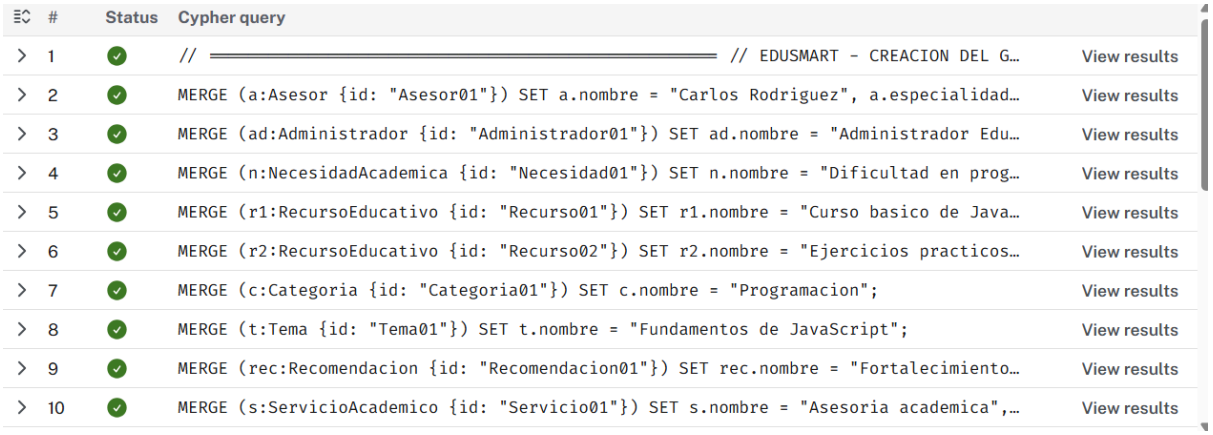
La implementación propuesta contempla diez nodos, catorce relaciones y tres consultas Cypher, incluyendo una consulta orientada a obtener conocimiento derivado. Los resultados y las evidencias definitivas deberán corresponder a la ejecución realizada en Neo4j.

4.8.8 CAPTURAS DE EVIDENCIA DE RELACIONES NODOS Y CYPHER

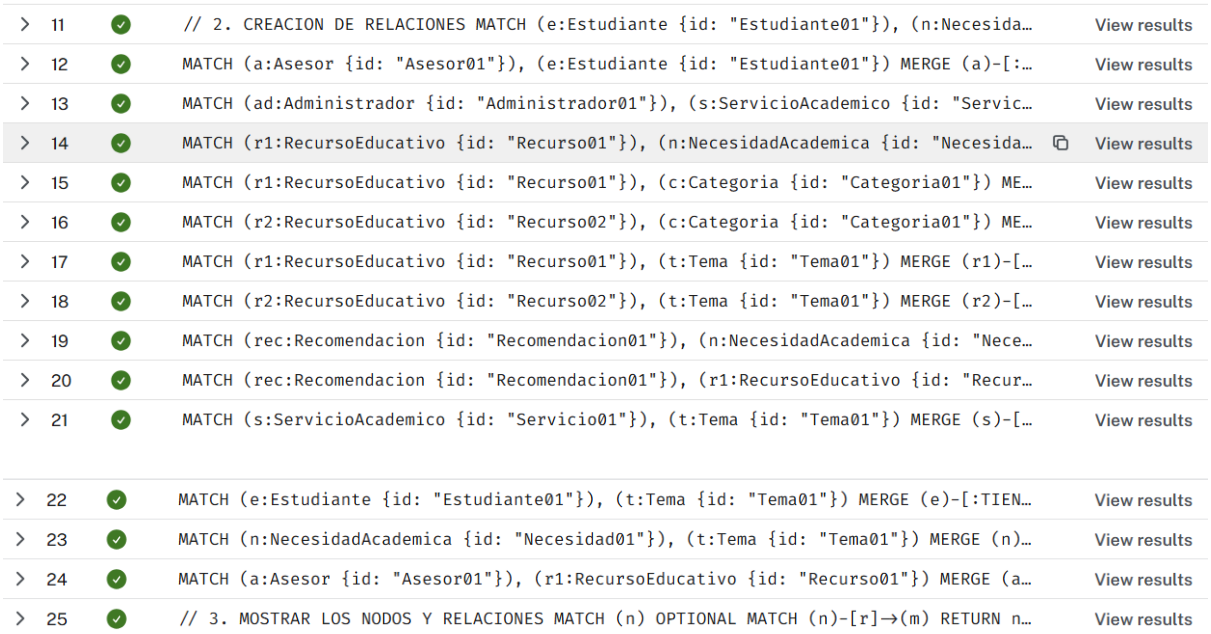
- verificación de que el programa funcione:



- archivo de consultas:



#	Status	Cypher query	
1	✓	// ═══ // EDUSMART - CREACION DEL G…	View results
2	✓	MERGE (a:Asesor {id: "Asesor01"}) SET a.nombre = "Carlos Rodriguez", a.especialidad…	View results
3	✓	MERGE (ad:Administrador {id: "Administrador01"}) SET ad.nombre = "Administrador Edu…	View results
4	✓	MERGE (n:NecesidadAcademica {id: "Necesidad01"}) SET n.nombre = "Dificultad en prog…	View results
5	✓	MERGE (r1:RecursoEducativo {id: "Recurso01"}) SET r1.nombre = "Curso basico de Java…	View results
6	✓	MERGE (r2:RecursoEducativo {id: "Recurso02"}) SET r2.nombre = "Ejercicios practicos…	View results
7	✓	MERGE (c:Categoria {id: "Categoria01"}) SET c.nombre = "Programacion";	View results
8	✓	MERGE (t:Tema {id: "Tema01"}) SET t.nombre = "Fundamentos de JavaScript";	View results
9	✓	MERGE (rec:Recomendacion {id: "Recomendacion01"}) SET rec.nombre = "Fortalecimiento…	View results
10	✓	MERGE (s:ServicioAcademico {id: "Servicio01"}) SET s.nombre = "Asesoria academica",…	View results



Codigo:

```
// ====================================================

// EDUSMART - CREACION DEL GRAFO DE CONOCIMIENTO

// 10 nodos y 14 relaciones

// ====================================================


// 1. CREACION DE NODOS


MERGE (e:Estudiante {id: "Estudiante01"})

SET e.nombre = "Laura Martinez",

    e.programa = "Ingenieria de Software";


MERGE (a:Asesor {id: "Asesor01"})

SET a.nombre = "Carlos Rodriguez",

    a.especialidad = "Orientacion academica";
```

```
MERGE (ad:Administrador {id: "Administrador01"})
SET ad.nombre = "Administrador EduSmart";

MERGE (n:NecesidadAcademica {id: "Necesidad01"})
SET n.nombre = "Dificultad en programacion",
    n.descripcion = "Fortalecer conocimientos de programacion";

MERGE (r1:RecursoEducativo {id: "Recurso01"})
SET r1.nombre = "Curso basico de JavaScript",
    r1.nivelDificultad = "Basico",
    r1.activo = true;

MERGE (r2:RecursoEducativo {id: "Recurso02"})
SET r2.nombre = "Ejercicios practicos de JavaScript",
    r2.nivelDificultad = "Intermedio",
    r2.activo = true;

MERGE (c:Categoria {id: "Categoria01"})
SET c.nombre = "Programacion";

MERGE (t:Tema {id: "Tema01"})
SET t.nombre = "Fundamentos de JavaScript";
```

```
MERGE (rec:Recomendacion {id: "Recomendacion01"})
SET rec.nombre = "Fortalecimiento de programacion",
    rec.descripcion = "Recomendacion de recursos educativos";

MERGE (s:ServicioAcademico {id: "Servicio01"})
SET s.nombre = "Asesoria academica",
    s.descripcion = "Acompanamiento academico";

// 2. CREACION DE RELACIONES

MATCH (e:Estudiante {id: "Estudiante01"}),
      (n:NecesidadAcademica {id: "Necesidad01"})
MERGE (e)-[:PRESENTA_NECESIDAD]->(n);

MATCH (a:Asesor {id: "Asesor01"}),
      (e:Estudiante {id: "Estudiante01"})
MERGE (a)-[:ORIENTA_A]->(e);

MATCH (ad:Administrador {id: "Administrador01"}),
      (s:ServicioAcademico {id: "Servicio01"})
MERGE (ad)-[:OFRECE_SERVICIO]->(s);

MATCH (r1:RecursoEducativo {id: "Recurso01"}),
      (n:NecesidadAcademica {id: "Necesidad01"})
```

```
MERGE (r1)-[:ATIENDE_NECESIDAD]->(n);

MATCH (r1:RecursoEducativo {id: "Recurso01"}),
      (c:Categoria {id: "Categoria01"})
MERGE (r1)-[:PERTENECE_A_CATEGORIA]->(c);

MATCH (r2:RecursoEducativo {id: "Recurso02"}),
      (c:Categoria {id: "Categoria01"})
MERGE (r2)-[:PERTENECE_A_CATEGORIA]->(c);

MATCH (r1:RecursoEducativo {id: "Recurso01"}),
      (t:Tema {id: "Tema01"})
MERGE (r1)-[:ABORDA_TEMA]->(t);

MATCH (r2:RecursoEducativo {id: "Recurso02"}),
      (t:Tema {id: "Tema01"})
MERGE (r2)-[:ABORDA_TEMA]->(t);

MATCH (rec:Recomendacion {id: "Recomendacion01"}),
      (n:NecesidadAcademica {id: "Necesidad01"})
MERGE (rec)-[:RESPONDE_A_NECESIDAD]->(n);

MATCH (rec:Recomendacion {id: "Recomendacion01"}),
      (r1:RecursoEducativo {id: "Recurso01"})
```

```
MERGE (rec)-[:RECOMIENDA_RECURSO]->(r1);

MATCH (s:ServicioAcademico {id: "Servicio01"}),
      (t:Tema {id: "Tema01"})
MERGE (s)-[:RELACIONADO_CON_TEMA]->(t);

MATCH (e:Estudiante {id: "Estudiante01"}),
      (t:Tema {id: "Tema01"})
MERGE (e)-[:TIENE_INTERES_EN]->(t);

MATCH (n:NecesidadAcademica {id: "Necesidad01"}),
      (t:Tema {id: "Tema01"})
MERGE (n)-[:TIENE_TEMA]->(t);

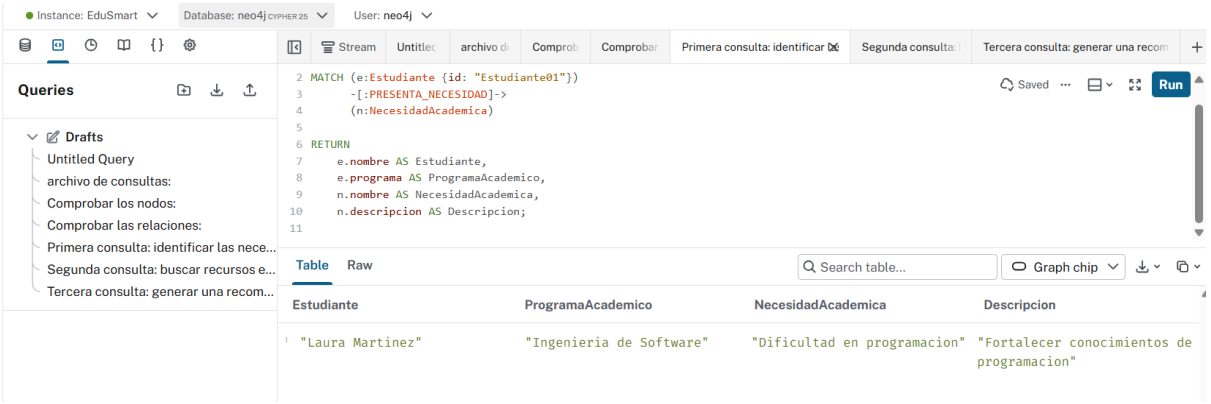
MATCH (a:Asesor {id: "Asesor01"}),
      (r1:RecursoEducativo {id: "Recurso01"})
MERGE (a)-[:ORIENTA_RECURSO]->(r1);

// 3. MOSTRAR LOS NODOS Y RELACIONES

MATCH (n)
OPTIONAL MATCH (n)-[r]->(m)
RETURN n, r, m;
```

CONSULTAS:

-consulta 1:

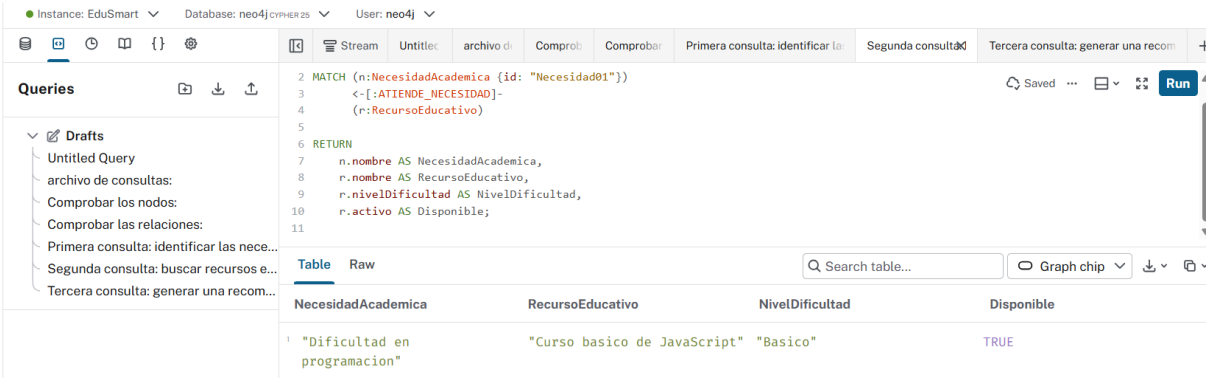


codigo:

```
MATCH (e:Estudiante {id: "Estudiante01"})
-[:PRESENTA_NECESIDAD]->
(n:NecesidadAcademica)

RETURN
e.nombre AS Estudiante,
e.programa AS ProgramaAcademico,
n.nombre AS NecesidadAcademica,
n.descripcion AS Descripcion;
```

-consulta 2:

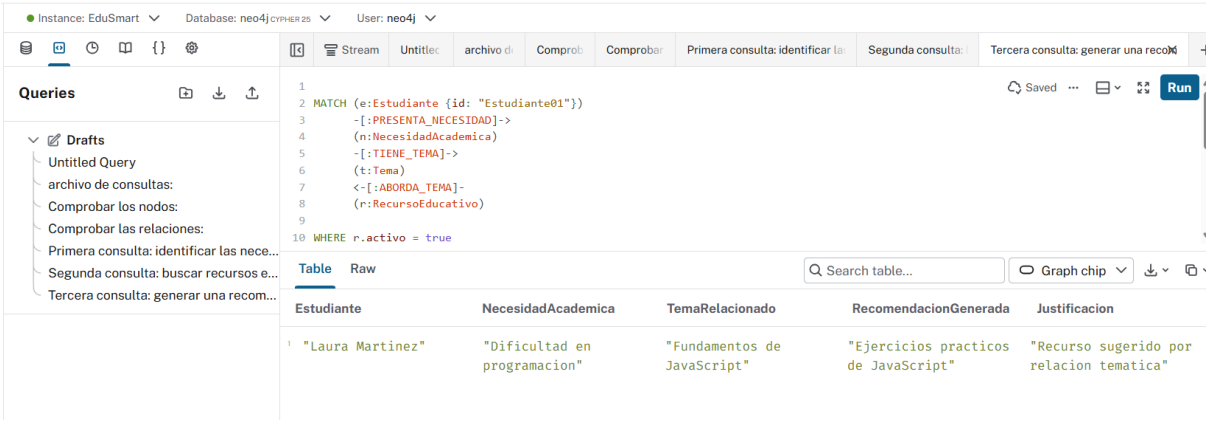


codigo:

```
MATCH (n:NecesidadAcademica {id: "Necesidad01"})
<-[:ATIENDE_NECESIDAD]-
(r:RecursoEducativo)

RETURN
n.nombre AS NecesidadAcademica,
r.nombre AS RecursoEducativo,
r.nivelDificultad AS NivelDificultad,
r.activo AS Disponible;
```

-consulta 3:





Código:

```
MATCH (e:Estudiante {id: "Estudiante01"})

-[:PRESENTA_NECESIDAD]->

(n:NecesidadAcademica)

-[:TIENE_TEMA]->

(t:Tema)

<-[:ABORDA_TEMA]-

(r:RecursoEducativo)


WHERE r.activo = true


AND NOT EXISTS {

MATCH (r)-[:ATIENDE_NECESIDAD]->(n)

}


RETURN

e.nombre AS Estudiante,

n.nombre AS NecesidadAcademica,
```

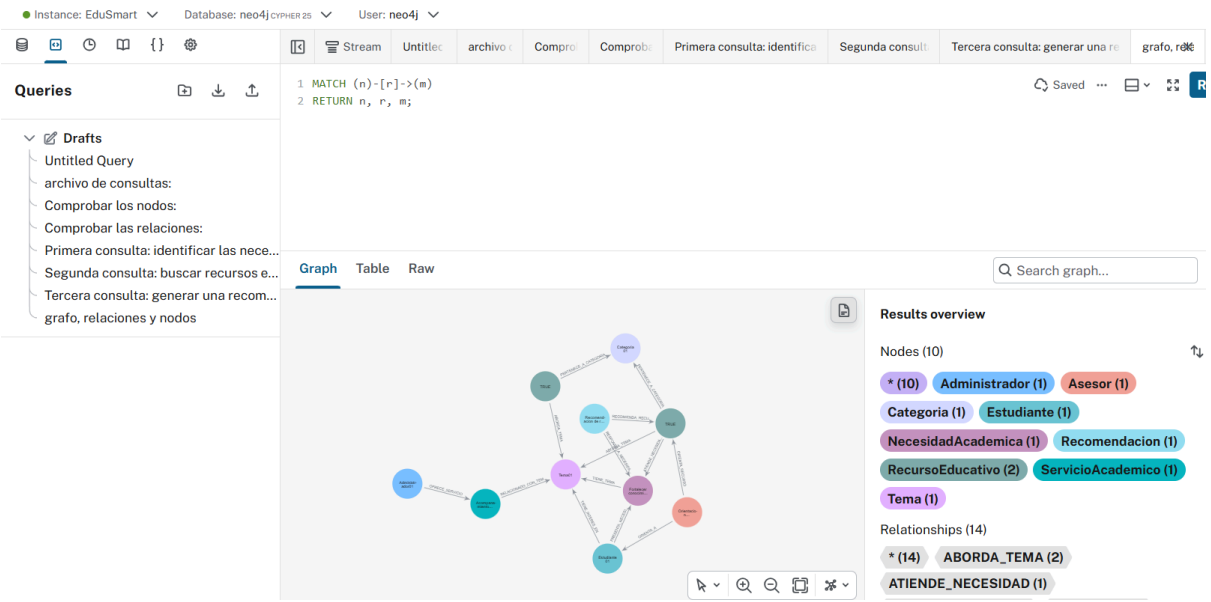
```
t.nombre AS TemaRelacionado,

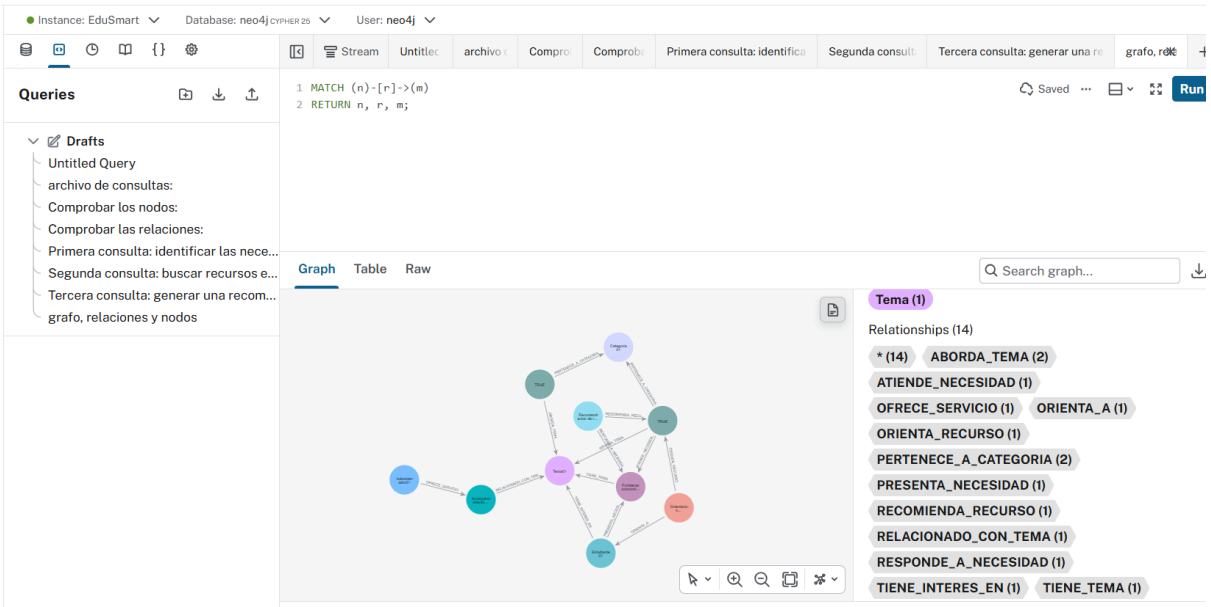
r.nombre AS RecomendacionGenerada,

"Recurso sugerido por relacion tematica"

AS Justificacion;
```

VERIFICACIÓN GRAFO.cypher:





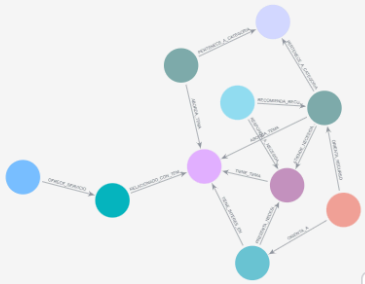
All results

```
// 3. MOSTRAR LOS NODOS Y RELACIONES MATCH (n) OPTIONAL MATCH (n)-[r]→(m) RETURN n, r, m
```

25 / 25

Graph | Table | Raw

Search graph...



Results overview

Nodes (10)

* (10) | Administrador (1) | Asesor (1) | Categoria (1) | Estudiante (1) | NecesidadAcademica (1) | Recomendacion (1) | RecursoEducativo (2) | ServicioAcademico (1) | Tema (1)

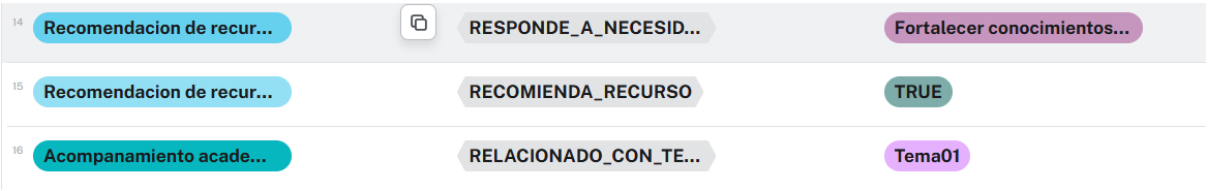
Relationships (14)

* (14) | ABORDA_TEMA (2)

Graph | Table | Raw

Search table... | Graph chip

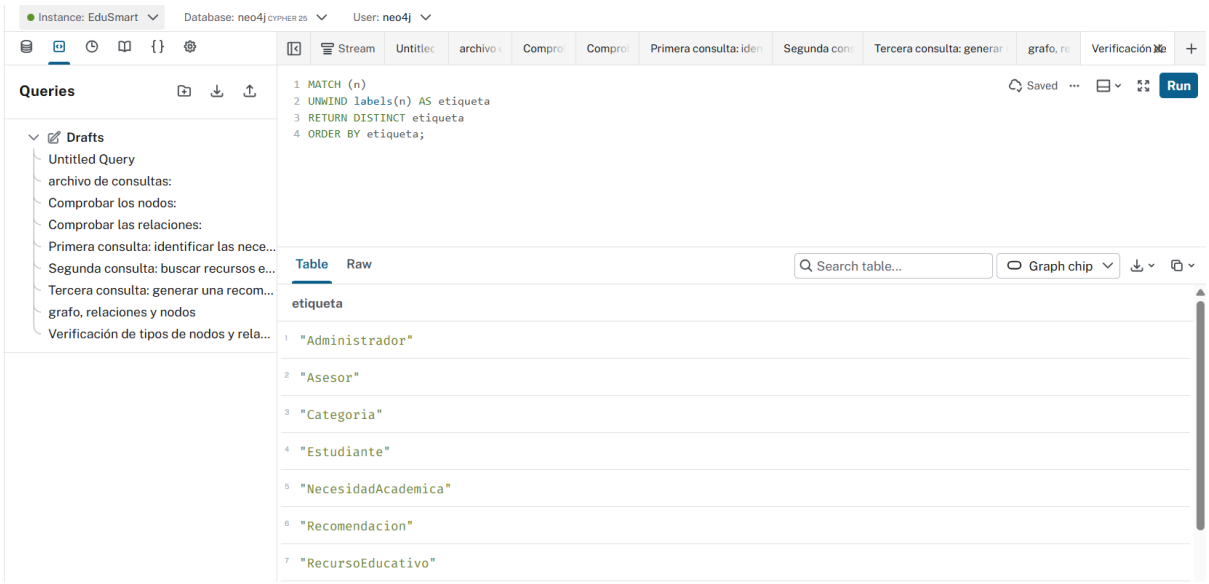
	n	r	m
1	Estudiante01	PRESENTA_NECESIDAD	Fortalecer conocimientos...
2	Estudiante01	TIENE_INTERES_EN	Tema01
3	Orientacion academica	ORIENTA_A	Estudiante01
4	Orientacion academica	ORIENTA_RECURSO	TRUE
5	Administrador01	OFRECE_SERVICIO	Acompanamiento acade...
6	Fortalecer conocimientos...	TIENE_TEMA	Tema01
7	TRUE	ATIENDE_NECESIDAD	Fortalecer conocimientos...
8	TRUE	PERTENECE_A_CATEGO...	Categoria01
9	TRUE	ABORDA_TEMA	Tema01
10	TRUE	PERTENECE_A_CATEGO...	Categoria01
11	TRUE	ABORDA_TEMA	Tema01
12	Categoria01	null	null
13	Tema01	null	null
14	Recomendacion de recur...	RESPONDE_A_NECESID...	Fortalecer conocimientos...
15	Recomendacion de recur...	RECOMIENDA_RECURSO	TRUE



Código:

```
MATCH (n)-[r]->(m)
RETURN n, r, m;
```

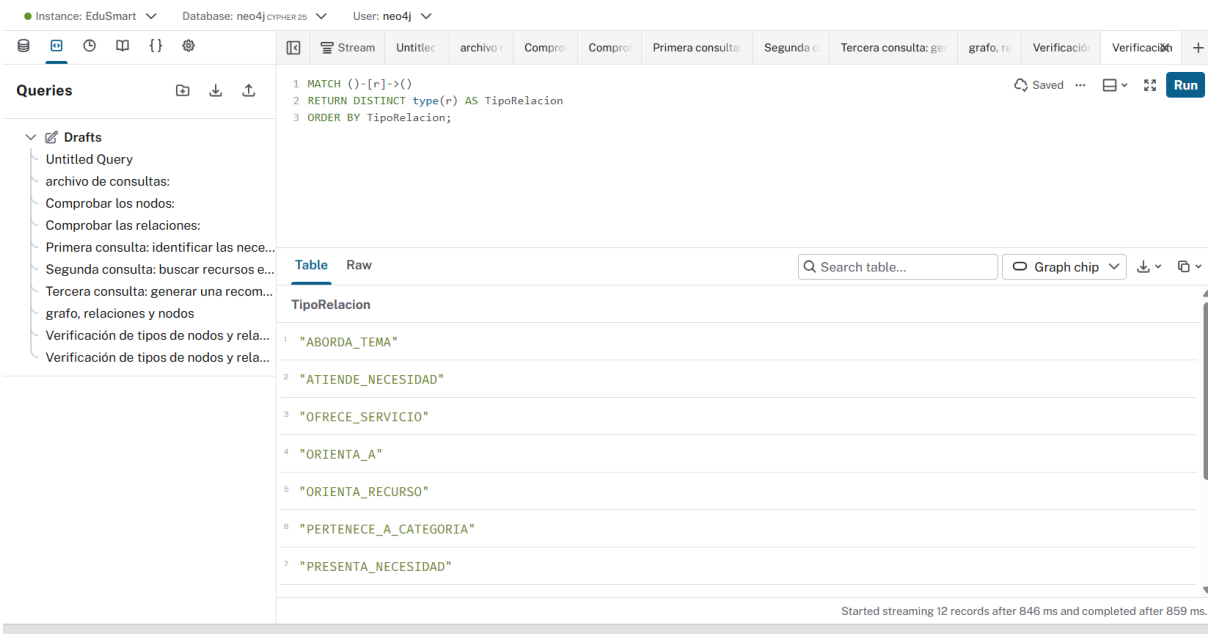
Verificación de tipos de nodos y relaciones 1:



código:

```
MATCH (n)
UNWIND labels(n) AS etiqueta
RETURN DISTINCT etiqueta
ORDER BY etiqueta;
```

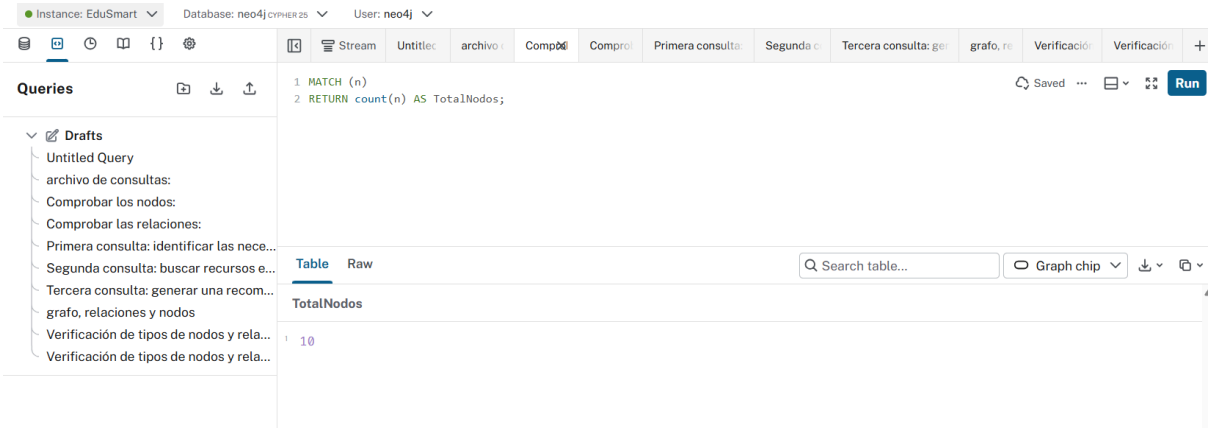
Verificación de tipos de nodos y relaciones 2 :



Codigo:

```
MATCH ()-[r]->()
RETURN DISTINCT type(r) AS TipoRelacion
ORDER BY TipoRelacion;
```

comprobar los nodos:

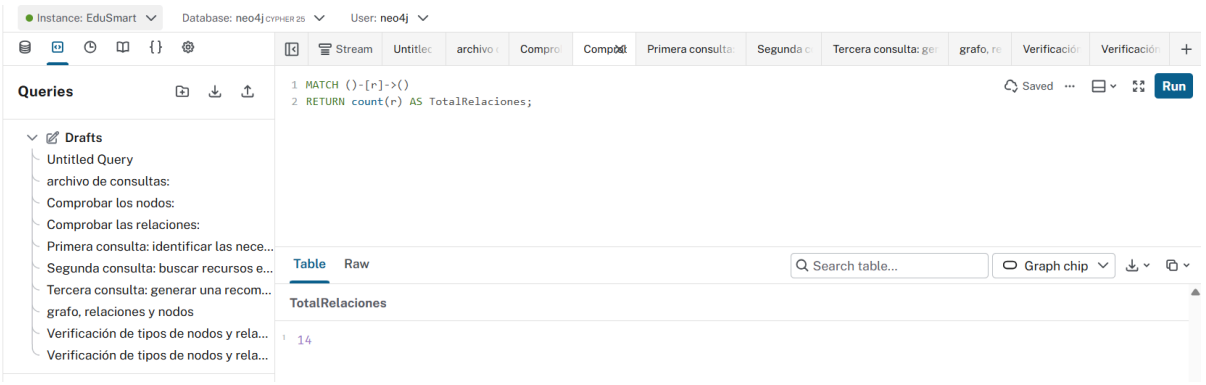


código:

```
MATCH (n)
```

```
RETURN count(n) AS TotalNodos;
```

comprobar las relaciones:



código:

```
MATCH ()-[r]->()
RETURN count(r) AS TotalRelaciones;
```

Base de datos informacion


Instance: EduSmart
Database: neo
Database information
Nodes (10)
*
Administrador
Asesor
Categoria
Estudiante
NecesidadAcademica
Recomendacion
RecursoEducativo
ServicioAcademico
Tema
Relationships (14)
*
ABORDA_TEMA
ATIENDE_NECESIDAD
OFRECE_SERVICIO
ORIENTA_A
ORIENTA_RECURSO
PERTENECE_A_CATEGORIA
PRESENTA_NECESIDAD
RECOMIENDA_RECURSO
RELACIONADO_CON_TEMA
RESPONDE_A_NECESIDAD
TIENE_INTERES_EN
TIENE_TEMA

Property keys

activo descripcion

especialidad id

nivelDificultad nombre

programa